

Praktikum 2: Laporan Praktikum 2 Machine Learning

Maryam Hasnaa' Syamila - 0110222067

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

E-mail: hasyamil4@gmail.com

Link Notebook Colab:  Praktikum02.ipynb

Link GitHub:

<https://github.com/maryamhasnaasyamila/machine-learning/tree/praktikum/praktikum/praktikum02/>

1. Sinkronisasi dengan Drive

Langkah pertama adalah menghubungkan *environment* Google Colab dengan Google Drive. Tujuannya adalah agar *notebook* Colab dapat mengakses file dataset yang tersimpan di dalam akun Drive

```
# menghubungkan colab dengan google drive
from google.colab import drive
drive.mount('/content/gdrive')
```

 Mounted at /content/gdrive

Proses ini berhasil dengan output **Mounted at /content/gdrive**, menandakan direktori Drive sudah siap diakses.

2. Membaca file csv dengan pandas

Setelah Drive terhubung, langkah selanjutnya adalah membaca file **500_Person_Gender_Height_Weight_Index.csv** menggunakan *library* Pandas.

```
# membaca file csv menggunakan pandas
import pandas as pd

df = pd.read_csv('/content/gdrive/MyDrive/SEMESTER 7/Machine Learning/praktikum02/datasets/500_Person_Gender_Height_Weight_Index.csv')
df
```



	Gender	Height	Weight	Index
0	Male	174	96	4
1	Male	189	87	2
2	Female	185	110	4
3	Female	195	104	3
4	Male	149	61	3
...	...	...	...	...
495	Female	150	153	5
496	Female	184	121	4
497	Female	141	136	5
498	Male	150	95	5
499	Male	173	131	5

500 rows x 4 columns

- Fungsi `pd.read_csv()` digunakan untuk memuat data dari file CSV ke dalam sebuah struktur data yang disebut DataFrame.

- DataFrame `df` ini pada dasarnya adalah sebuah tabel yang berisi seluruh data dari file CSV, yang terdiri dari 500 baris dan 4 kolom. Kolom-kolom tersebut adalah 'Gender', 'Height', 'Weight', dan 'Index'.

3. Melihat Info Data

```
# mencari info data pada file (tipe datanya, non null count data, nama kolom)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype
---  -
0   Gender  500 non-null      object
1   Height  500 non-null      int64
2   Weight  500 non-null      int64
3   Index   500 non-null      int64
dtypes: int64(3), object(1)
memory usage: 15.8+ KB
```

Fungsi ini memberikan ringkasan teknis dari DataFrame:

- Terdapat 500 entri data, dari indeks 0 hingga 499.
- Terdapat 4 kolom: 'Gender', 'Height', 'Weight', dan 'Index'.
- Semua kolom memiliki 500 data non-null, artinya tidak ada data yang kosong atau hilang.
- Kolom 'Gender' bertipe `object` (teks), sementara sisanya bertipe `int64` (angka bulat).

4. Mencari nilai Mean (nilai rata-rata)

Analisis statistik dimulai dengan mencari nilai rata-rata (mean) dari kolom 'Height'. Mean berguna untuk mengetahui nilai pusat dari data.

```
[ ] # menghitung mean semua kolom numerik
    df['Height'].mean()

np.float64(169.944)
```

Diperoleh nilai mean untuk tinggi badan sebesar **169.944**. Ini berarti, rata-rata tinggi badan dari 500 orang dalam dataset ini adalah sekitar 170 cm.

5. Mencari nilai Median (nilai tengah)

Median dicari untuk menemukan nilai tengah dari data tinggi badan setelah diurutkan. Median lebih kebal terhadap nilai ekstrem (outlier) jika dibandingkan dengan mean.

```
[ ] # menghitung median semua kolom numerik
    df['Height'].median()

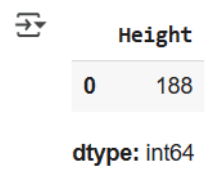
170.5
```

Nilai median dari kolom 'Height' adalah **170.5**. Artinya, 50% dari data tinggi badan berada di bawah 170.5 cm, dan 50% sisanya berada di atasnya.

6. Mencari nilai Modus (nilai yang paling sering muncul)

Modus digunakan untuk mengetahui nilai manakah yang memiliki frekuensi kemunculan tertinggi dalam data tinggi badan.

```
[ ] # menghitung modus
df['Height'].mode()
```



Height

0	188
---	-----

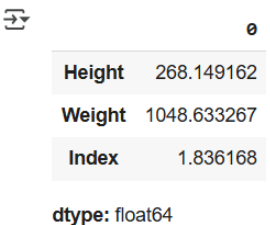
dtype: int64

Nilai modus yang didapatkan adalah **188**. Ini menunjukkan bahwa tinggi badan 188 cm adalah yang paling banyak ditemukan dalam dataset ini.

7. Menghitung Variansi

Variansi dihitung untuk mengukur seberapa tersebar data dari nilai rata-ratanya. Semakin besar nilainya, semakin bervariasi datanya

```
[ ] # menghitung variansi
df.var(numeric_only=True)
```



0

Height	268.149162
Weight	1048.633267
Index	1.836168

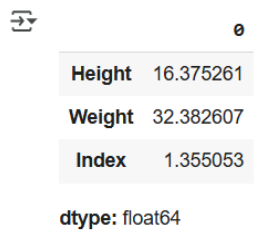
dtype: float64

Diperoleh nilai variansi untuk setiap kolom numerik: 'Height' (268.149), 'Weight' (1048.633), dan 'Index' (1.836).

8. Menghitung Standar Deviasi

Standar deviasi (simpangan baku) juga merupakan ukuran sebaran data, namun dalam satuan yang sama dengan data aslinya (akar kuadrat dari variansi), sehingga lebih mudah diinterpretasikan.

```
[ ] # menghitung standar deviasi
df.std(numeric_only=True)
```



0

Height	16.375261
Weight	32.382607
Index	1.355053

dtype: float64

Nilai standar deviasi untuk 'Height' adalah sekitar **16.37**, 'Weight' **32.38**, dan 'Index' **1.35**.

9. Menghitung Q1, Q3, dan IQR

Langkah ini bertujuan untuk membagi data menjadi empat bagian sama besar untuk memahami distribusinya lebih dalam.

- **Q1 (Kuartil 1):** Nilai yang membatasi 25% data terendah.
- **Q3 (Kuartil 3):** Nilai yang membatasi 75% data terendah.
- **IQR (Interquartile Range):** Jangkauan antara Q3 dan Q1, yang menunjukkan sebaran 50% data di bagian tengah.

```
[ ]  
  
# hitung kuartil pertama (Q1)  
q1 = df['Height'].quantile(0.25)  
print("Q1: ", q1)  
  
# hitung kuartil ketiga (Q3)  
q3 = df['Height'].quantile(0.75)  
print("Q3: ", q3)  
  
# hitung IQR (Interquartile Range)  
iqr = q3 - q1  
print("IQR: ", iqr)
```

```
Q1: 156.0  
Q3: 184.0  
IQR: 28.0
```

Diperoleh Q1 = **156.0** , Q3 = **184.0** , dan IQR = **28.0**.

10. Membuat Deskripsi Statistika

Fungsi `describe()` digunakan untuk menampilkan rangkuman lengkap statistik deskriptif dari semua kolom numerik secara otomatis.

```
[ ]  
  
# untuk membuat statistika deskripsi pada type data int  
df.describe()
```

```
Height      Weight      Index  
count  500.000000  500.000000  500.000000  
mean   169.944000  106.000000   3.748000  
std     16.375261  32.382607   1.355053  
min     140.000000  50.000000   0.000000  
25%    156.000000  80.000000   3.000000  
50%    170.500000  106.000000  4.000000  
75%    184.000000  136.000000  5.000000  
max     199.000000  160.000000  5.000000
```

Outputnya berupa tabel yang merangkum jumlah data (**count**), rata-rata (**mean**), standar deviasi (**std**), nilai minimum (**min**), kuartil (25%, 50%, 75%), dan nilai maksimum (**max**) untuk kolom 'Height', 'Weight', dan 'Index'.

11. Menghitung dan menampilkan Matriks Korelasi

Matriks korelasi dibuat untuk mengukur kekuatan dan arah hubungan linear antar variabel numerik. Nilai korelasi berkisar dari -1 (berbanding terbalik) hingga +1 (berbanding lurus).

```
[ ]  
  
# menghitung matriks korelasi semua kolom numerik  
correlation_matrix = df.corr(numeric_only=True)  
  
# menampilkan matriks korelasi  
print("Matriks Korelasi:")  
print(correlation_matrix)
```

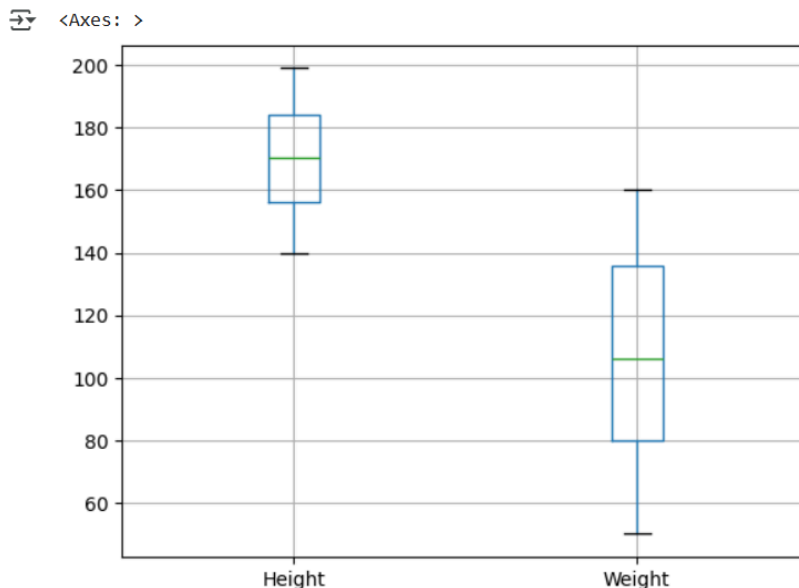
```
↗ Matriks Korelasi:  
      Height  Weight  Index  
Height 1.000000 0.000446 -0.422223  
Weight 0.000446 1.000000 0.804569  
Index -0.422223 0.804569 1.000000
```

Dari matriks korelasi, terlihat bahwa ada hubungan positif yang kuat antara 'Weight' dan 'Index' dengan nilai **0.804569**. Sebaliknya, hubungan antara 'Height' dan 'Index' bersifat negatif sedang (-0.422223).

12. Membuat visualisasi data dengan Boxplot

Boxplot digunakan untuk memvisualisasikan distribusi data 'Height' dan 'Weight' secara grafis. Grafik ini efektif untuk melihat sebaran data, median, kuartil, dan potensi adanya outlier.

```
[ ]  
  
import pandas as pd  
import numpy as np  
  
df.boxplot(column=['Height', 'Weight'])
```



Visualisasi boxplot menunjukkan ringkasan statistik dari kedua kolom. Dari grafik tersebut, kita bisa membandingkan rentang nilai dan sebaran data tinggi badan dan berat badan secara visual.

13. Membuat visualisasi data dengan Histogram

Histogram dibuat khusus untuk kolom 'Height' untuk melihat distribusi frekuensinya. Grafik ini menunjukkan seberapa sering rentang nilai tinggi badan tertentu muncul dalam data.

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# ambil data height
data_height = df["Height"]

# buat histogram
n, bins, patches = plt.hist(data_height, bins=5, color='pink', edgecolor='black')

# tambahkan label
plt.title('Histogram Nilai')
plt.xlabel('Height')
plt.ylabel('Frekuensi')

# tampilkan rentang frekuensi di sumbu x
bin_centers = 0.5*(bins[:-1] + bins[1:])
plt.xticks(bin_centers, ['{:0f}-{:0f}'.format(bins[i], bins[i+1]) for i in range(len(bins)-1)])

# tampilkan histogram
plt.show
```

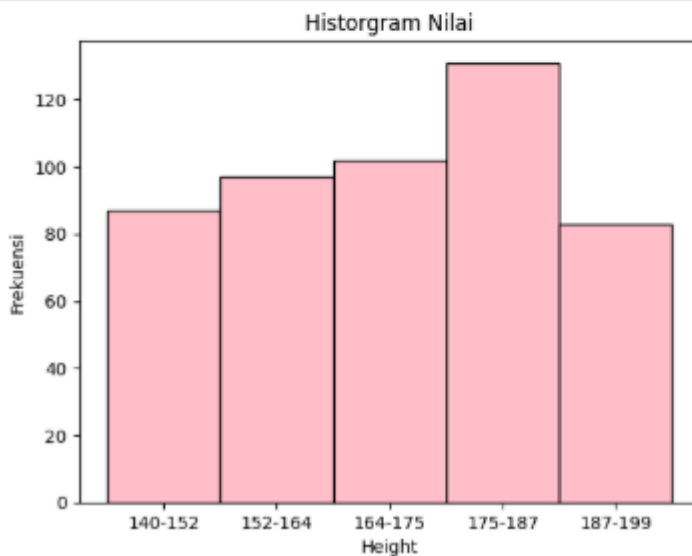


```
matplotlib.pyplot.show
def show(*args, **kwargs) -> None
```

Display all open figures.

Parameters

block : bool, optional
Whether to wait for all figures to be closed before returning.



Histogram yang dihasilkan menunjukkan bahwa rentang tinggi badan antara **175-187 cm** memiliki frekuensi tertinggi, yaitu lebih dari 120 data.

14. Membuat visualisasi data dengan Scatter Plot menggunakan data Nilai1 dan Nilai2 untuk Korelasi Positif

Scatter plot digunakan untuk memvisualisasikan hubungan antara dua variabel. Pada praktikum ini, dibuat dua contoh scatter plot menggunakan data buatan untuk mengilustrasikan korelasi positif dan negatif.

```

import pandas as pd
import matplotlib.pyplot as plt

# buat dataframe contoh
data = {
    'Nilai1': [1,2,3,4,5,6,7,8,9,10],
    'Nilai2': [2,4,6,8,10,12,14,16,18,20]
}

df2 = pd.DataFrame(data)

# buat scatter plot
plt.scatter(df2['Nilai1'], df2['Nilai2'], color='blue', marker='o')

# tambahkan label
plt.title('Scatter Plot Korelasi Positif')
plt.xlabel('Nilai1')
plt.ylabel('Nilai2')

# tambahkan grid
plt.grid(True)

# tampilkan plot
plt.show

```

(4)

```

matplotlib.pyplot.show
def show(*args, **kwargs) -> None

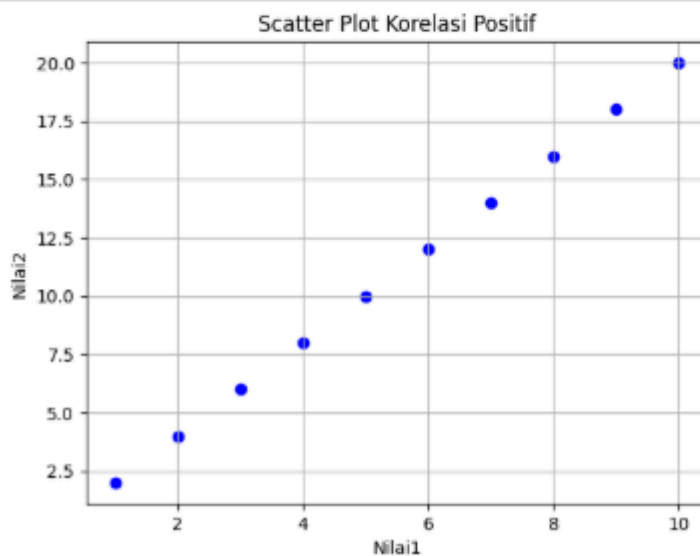
```

Display all open figures.

Parameters

block : bool, optional

Whether to wait for all figures to be closed before returning.



Scatter Plot Korelasi Positif: Grafik menunjukkan pola titik yang naik dari kiri bawah ke kanan atas. Ini mengindikasikan bahwa saat 'Nilai1' bertambah, 'Nilai2' juga ikut bertambah.

15. Membuat visualisasi data dengan Scatter Plot menggunakan Nilai1 dan Nilai2 untuk Korelasi Negatif

```
import pandas as pd
import matplotlib.pyplot as plt

# buat dataframe contoh
data = {
    'Nilai1': [1,2,3,4,5,6,7,8,9,10],
    'Nilai2': [10,9,8,7,6,5,4,3,2,1]
}

df3 = pd.DataFrame(data)

# buat scatter plot
plt.scatter(df3['Nilai1'], df3['Nilai2'], color='red', marker='x')

# tambahkan label
plt.title('Scatter Plot Korelasi Negatif')
plt.xlabel('Nilai1')
plt.ylabel('Nilai2')

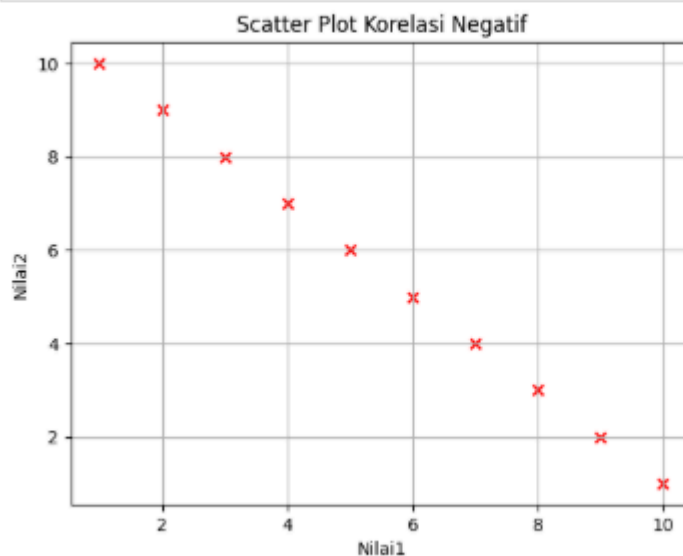
# tambahkan grid
plt.grid(True)

# tampilkan plot
plt.show
```

```
matplotlib.pyplot.show
def show(*args, **kwargs) -> None

Display all open figures.

Parameters
-----
block : bool, optional
    Whether to wait for all figures to be closed before returning.
```



Scatter Plot Korelasi Negatif: Grafik ini menunjukkan pola titik yang menurun dari kiri atas ke kanan bawah. Ini berarti saat 'Nilai1' bertambah, 'Nilai2' justru berkurang.